

Parte 2: Introducción a Wireshark

Redes de Computadoras - CC308

José Antonio Mérida Castejón - 201105

13 de julio de 2026

Introducción

En la primera parte de este laboratorio se replicaron esquemas de comunicación previos a las redes modernas (código Morse, código Baudot y un protocolo propio de conmutación de mensajes), lo cual permitió evidenciar de forma práctica los problemas de fiabilidad, ergonomía y enrutamiento que enfrentaban dichos esquemas. En esta segunda parte se aborda una herramienta central para el estudio de las redes de computadoras actuales: Wireshark, un analizador de protocolos que permite capturar y examinar el tráfico que circula por una interfaz de red en tiempo real.

El objetivo de esta parte es familiarizarse con el entorno de Wireshark, personalizando perfiles, layout y filtros, para luego realizar una captura de paquetes sobre la interfaz de red activa de la computadora y analizar una solicitud HTTP real, identificando las versiones del protocolo utilizadas, los encabezados relevantes de la solicitud y de la respuesta, y discutiendo consideraciones de rendimiento y seguridad relacionadas con el uso de esta herramienta.

Personalización del Entorno

Configuración de perfil

Se creó un nuevo perfil en Wireshark para personalizar la captura sin afectar la configuración por defecto.

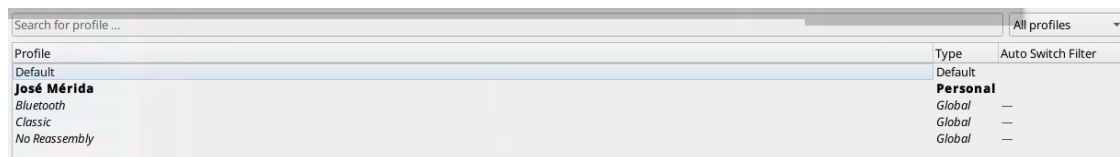


Figura 1: Creación del perfil personalizado en Wireshark.

Layout, Filtro y Botón de Filtro

Se modificó el layout de la interfaz, agregando un botón de filtro y una regla de coloreo personalizada. Adicionalmente, sobre el archivo `intro-wireshark-trace1.pcap` proporcionado se aplicó el formato de tiempo `Time of Day` y se agregó una columna con la longitud del protocolo, ocultando la columna `Longitud` original, cambios que también pueden observarse en la Figura 2.

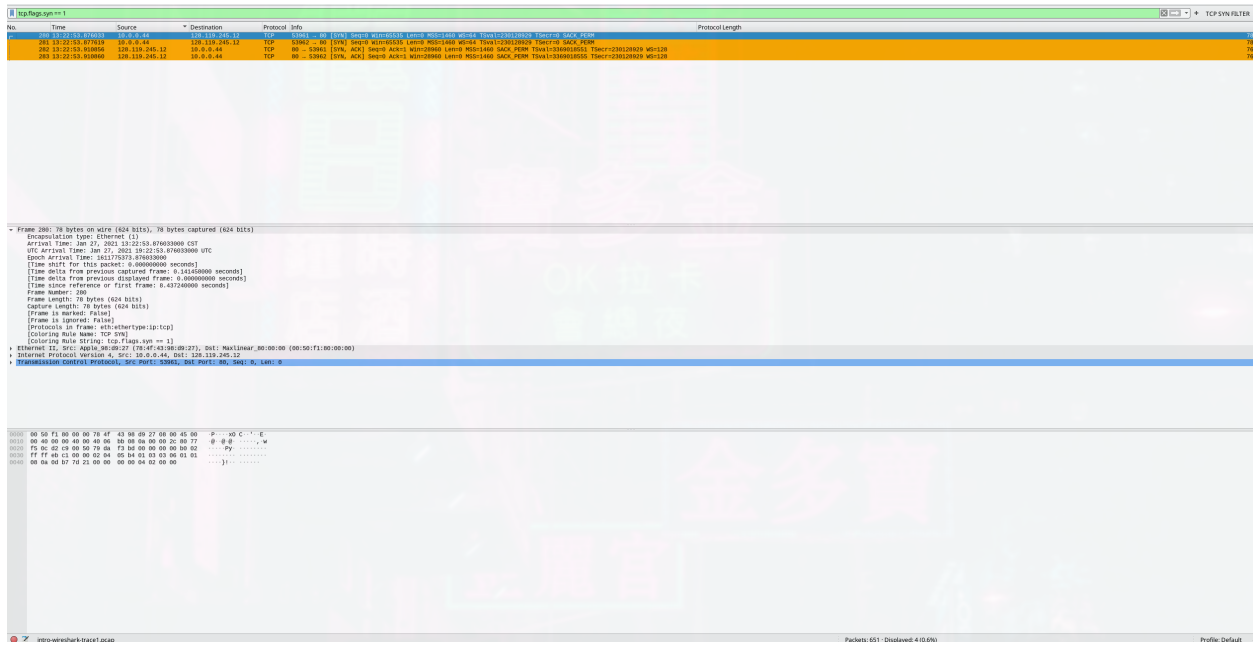


Figura 2: Cambio de layout, botón agregado y regla de coloreo.

Name	Filter
☒ TCP SYN	tcp.flags.syn == 1
☒ Bad TCP	tcp.analysis.flags && !tcp.analysis.window_update && !tcp.analysis.keep_alive && !tcp.analysis.keep_alive_ack
☒ HSRP State Change	hsrp.state != 8 && hsrp.state != 16
☒ Spanning Tree Topology Change	stp.type == 0x80
☒ OSPF State Change	ospf.msg != 1
☒ ICMP errors	icmp.type in { 3, 5, 11 } icmpv6.type in { 1, 4 }
☒ ARP	arp
☒ ICMP	icmp icmpv6
☒ TCP RST	tcp.flags.reset eq 1
☒ SCTP ABORT	sctp.chunk_type eq ABORT
☒ IPv4 TTL low or unexpected	(ip.dst != 224.0.0.0/4 && ip.ttl < 5 && !ospf !eigrp !bgp tcp.port == 179) (ip.dst == 224.0.0.0/24 && ip.dst != 224.0.0.251 && ip.ttl != 1 && !vrrp !carp !eigrp !rip !ospf !bgp !tcp.port == 179) (ip.v6.dst == ff00::/8 && ip.v6.hlim not in { 1, 64, 255 })
☒ IPv6 hop limit low or unexpected	(ip.v6.dst != ff00::/8 && ip.v6.hlim < 5 && !ospf !bgp !tcp.port == 179) (ip.v6.dst == ff00::/8 && ip.v6.hlim not in { 1, 64, 255 })
☒ Checksum Errors	eth.fcs.status == "Bad" ip.checksum.status == "Bad" tcp.checksum.status == "Bad" udp.checksum.status == "Bad" sctp.checksum.status == "Bad" mstp.checksum.status == "Bad"
☒ SMB	smb nbss nbns netbios
☒ HTTP	http tcp.port == 80 http2
☒ DCE/RPC	dcerpc
☒ Routing	hsrp eigrp ospf bgp cdp vrrp carp gvrp igmp ismp
☒ TCP SYN/FIN	tcp.flags & 0x02 tcp.flags.fin == 1
☒ TCP	tcp
☒ UDP	udp
☒ Broadcast	eth[0] & 1
☒ System Event	systemd_journal sysdig

Figura 3: Regla de coloreo personalizada agregada en Wireshark.

Interfaces Virtuales

Se removieron las interfaces virtuales o que no se encuentran en uso dentro del sistema, esto también se verifica por medio del comando `ifconfig` ejecutado en una sección posterior del laboratorio.

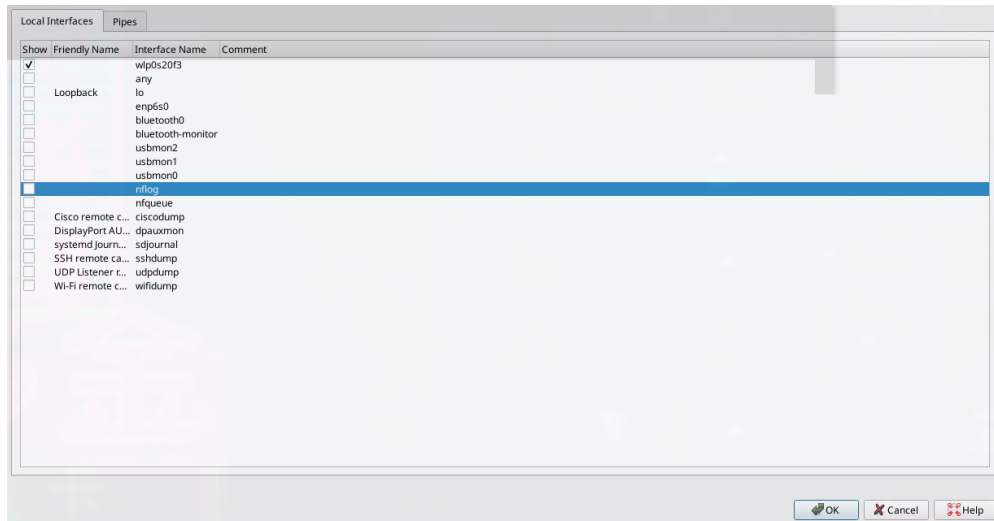


Figura 4: Listado de interfaces sin las interfaces virtuales.

Configuración de la Captura de Paquetes

```

) ifconfig
enp6s0: flags=4099<UP,BROADCAST,MULTICAST> mtu 1500
    ether 08:bf:b8:a0:11:50 txqueuelen 1000 (Ethernet)
    RX packets 0 bytes 0 (0.0 B)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 0 bytes 0 (0.0 B)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0
    device memory 0x82f00000-82ffffff

lo: flags=73<UP,LOOPBACK,RUNNING> mtu 65536
    inet 127.0.0.1 netmask 255.0.0.0
    inet6 ::1 prefixlen 128 scopeid 0x10<host>
    loop txqueuelen 1000 (Local Loopback)
    RX packets 1141 bytes 137838 (134.6 KiB)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 1141 bytes 137838 (134.6 KiB)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

wlp0s20f3: flags=4163<UP,BROADCAST,RUNNING,MULTICAST> mtu 1500
    inet 192.168.1.17 netmask 255.255.255.0 broadcast 192.168.1.255
    inet6 fe80::aa69:feb:4926:90d1 prefixlen 64 scopeid 0x20<link>
    inet6 2800:98:1124:aa7::1 prefixlen 128 scopeid 0x0<global>
    inet6 2800:98:1124:aa7:ff97:6b9e:b3c6:6e5a prefixlen 64 scopeid 0x0<global>
    ether 0e:62:e7:72:da:65 txqueuelen 1000 (Ethernet)
    RX packets 34769 bytes 34577387 (32.9 MiB)
    RX errors 0 dropped 1 overruns 0 frame 0
    TX packets 15481 bytes 7619843 (7.2 MiB)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

```

Figura 5: Ejecución de comando ifconfig.

El comando `ifconfig` muestra todos los adaptadores de red disponibles en la computadora, al igual que sus direcciones IP (en caso de tener una asignada) e ID's de hardware. En este caso podemos ver tres dispositivos:

- **enp6s0:** Interfaz de ethernet, no tiene IP asignada y no ha mandado o recibido ningún paquete. Esto es consistente con el hecho que no está conectado ningún cable de ethernet a la computadora al momento de ejecutar el comando.
- **lo:** Tiene asignada la IP `127.0.0.1`, es una interfaz virtual utilizada por el sistema operativo. Este es el localhost.

- **wlp0s20f3**: Interfaz de internet inalámbrica, se puede ver la IP asignada 192.168.1.17 al igual que los paquetes transmitidos (TX) y paquetes recibidos (RX) desde el momento de arranque. Esta es la interfaz de red activa del sistema.

Utilizando el dispositivo **wlp0s20f3** se realizó la captura con la siguiente configuración:

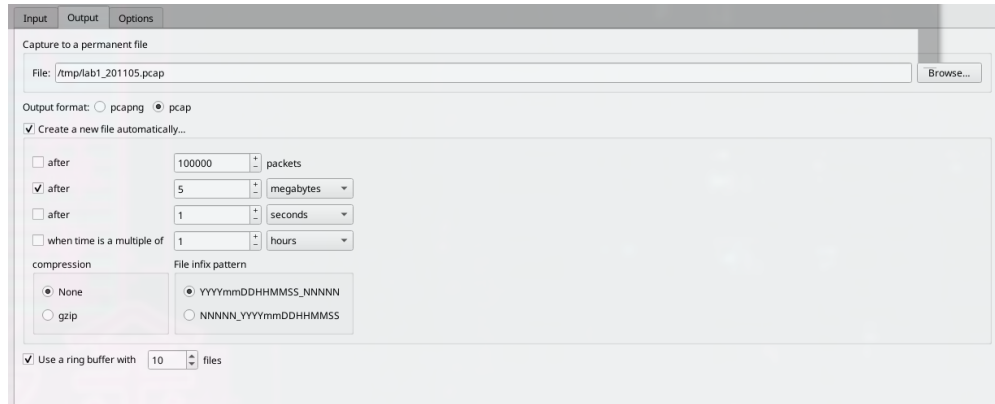


Figura 6: Configuración de las opciones de salida de la captura en Wireshark.

Los archivos resultantes se encuentran dentro del repositorio de la entrega, dentro del directorio **files**.

Análisis de Paquetes

Para el análisis se filtró el tráfico capturado utilizando el filtro **http**, aislando así únicamente las solicitudes y respuestas del protocolo HTTP generadas al cargar la página de prueba del laboratorio (**gaia.cs.umass.edu**).

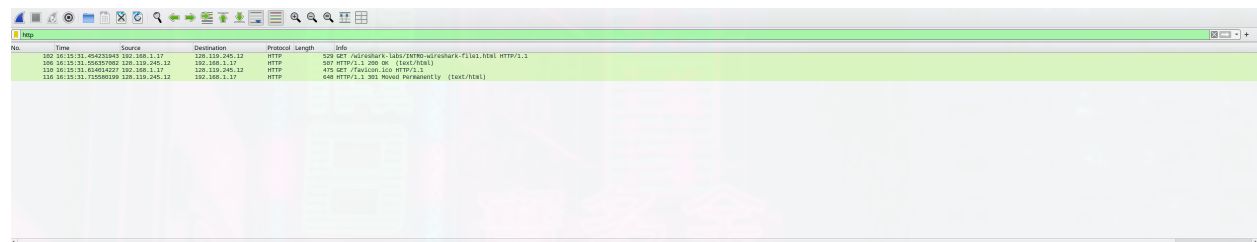


Figura 7: Solicitudes y respuestas HTTP capturadas al filtrar por **http**.

Preguntas

- ¿Qué versión de HTTP está ejecutando su navegador?
HTTP/1.1, evidenciado por la solicitud GET inicial hacia la página.
- ¿Qué versión de HTTP está ejecutando el servidor?
HTTP/1.1, evidenciado por la respuesta 200 OK del servidor.

- c. ¿Qué lenguajes (si aplica) indica el navegador que acepta a el servidor?

```
> GET /wireshark-labs/INTRO-wireshark-file1.html HTTP/1.1\r\n
Host: gaia.cs.umass.edu\r\n
Connection: keep-alive\r\n
Upgrade-Insecure-Requests: 1\r\n
User-Agent: Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/142.0.0 Safari/537.36\r\n
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7\r\n
Accept-Encoding: gzip, deflate\r\n
Accept-Language: en-US,en;q=0.9\r\n
\r\n
[Response in frame: 106]
[Full request URI: http://gaia.cs.umass.edu/wireshark-labs/INTRO-wireshark-file1.html]
```

Figura 8: Encabezados de la solicitud GET, incluyendo **Accept-Language**.

en-US, en. Inglés Americano e Inglés.

- d. ¿Cuántos bytes de contenido fueron devueltos por el servidor?

```
▼ Hypertext Transfer Protocol
  ▶ HTTP/1.1 200 OK\r\n
    Date: Mon, 13 Jul 2026 22:15:31 GMT\r\n
    Server: Apache/2.4.62 (AlmaLinux) OpenSSL/3.5.5 mod_fcgid/2.3.9 mod_perl/2.0.12 Perl/v5.32.1\r\n
    Last-Modified: Tue, 28 Oct 2025 05:59:01 GMT\r\n
    ETag: "51-64231b6715777"\r\n
    Accept-Ranges: bytes\r\n
    ▼ Content-Length: 81\r\n
      [Content length: 81]
    Keep-Alive: timeout=5, max=100\r\n
    Connection: Keep-Alive\r\n
    Content-Type: text/html; charset=UTF-8\r\n
    \r\n
    [Request in frame: 102]
```

Figura 9: Encabezados de la respuesta HTTP, incluyendo **Content-Length**.

81 bytes.

- e. En el caso que haya un problema de rendimiento mientras se carga la página, ¿en qué elementos de la red convendría escuchar los paquetes?

Conviene escuchar en varios puntos de la red y no solo en uno: el cliente, el servidor y los intermediarios como routers o proxies [4]. El retraso puede venir del servidor, de un enlace intermedio o del propio cliente, y comparando capturas de varios puntos a la vez es más fácil ver dónde se concentra realmente la latencia [1].

- f. ¿Es conveniente instalar Wireshark en el servidor? Justifique

No. Wireshark es pesado y necesita interfaz gráfica, y la mayoría de servidores de producción ni siquiera tienen una instalada. Los recursos del servidor deberían priorizarse para el servicio que ofrece, y agregar software extra solo suma superficie de ataque y posibles fallos. Por eso lo normal es usar algo más ligero como tcpdump para capturar en el servidor, y luego analizar esa captura en otra máquina con Wireshark [4].

Discusión General

Ambas partes del laboratorio coinciden en ser una introducción a los problemas fundamentales de la comunicación en red pero desde ángulos muy distintos. En la primera parte reproducimos “a mano” las dificultades de codificar, transmitir y decodificar un mensaje mediante Morse y Baudot, y diseñamos un protocolo propio de conmutación para varios clientes. Por otro lado, en esta segunda parte observamos directamente cómo esos mismos problemas (codificación, encabezados,

enrutamiento, fiabilidad) ya están resueltos y en funcionamiento dentro del tráfico real que generan un navegador y un servidor web al comunicarse mediante HTTP. En mi opinión, es bastante interesante poder observar los dos extremos siendo uno comunicación rudimentaria y otro los protocolos modernos que cuentan con herramientas e infraestructura para poder explorarlos.

Personalmente, me pareció valioso poder *ver* el protocolo en acción en vez de solo leer sobre su funcionamiento en teoría. Configurar el entorno de Wireshark (perfiles, layout, reglas de coloreo) ayudó a entender un poco más la herramienta, ya que puede ser un poco abrumante al iniciar e intentar llevar a cabo tareas más complejas. El filtrar por `http` y examinar los encabezados también sirvió como un ejemplo práctico de como se solucionan las problemáticas que tuvimos en la parte 1, es importante que el emisor y el receptor compartan un mismo “lenguaje” al comunicarse. En este caso, existen campos del encabezado específico para poder detallar información del mensaje y lo que se espera en el retorno.

También me pareció interesante la discusión sobre el monitoreo de la red, mientras más compleja sea la infraestructura también se requiere un monitoreo más extenso para asegurar la integridad del sistema a lo largo de sus diferentes componentes. Concluyendo, personalmente me pareció una buena introducción al curso. Logramos identificar algunas de las problemáticas que los protocolos de red modernos resuelven, al igual que una pequeña exposición a herramientas que se utilizan hoy en día y algunas convenciones del protocolo `http`.

Conclusiones

Este laboratorio me dejó claro que establecer un protocolo de comunicación desde cero es un problema bastante grande, incluso en un sistema pequeño. Trabajar con Morse y Baudot en la parte 1 mostró que cosas que hoy se dan por sentadas, como que el receptor sepa cuándo inicia y termina un mensaje o que ambas partes compartan el mismo código, requieren decisiones de diseño explícitas y no ocurren solas.

Revisar los encabezados de HTTP en la parte 2 me permitió confirmar que estos, que antes veía como detalles secundarios del desarrollo web, en realidad resuelven los mismos problemas que enfrentamos a mano con Morse y Baudot solo que de forma más depurada y estandarizada. Esto refuerza mi entendimiento detrás de las decisiones tomadas al desarrollarse los protocolos modernos.

Tanto pensar en la escalabilidad del conmutador de la parte 1 como la discusión sobre dónde conviene monitorear una red en la parte 2 me mostraron que la complejidad de un sistema de comunicación no depende solo del protocolo que se use, sino también de cómo este se comporta y se supervisa conforme crece. Por eso el diseño de un protocolo y la infraestructura a su alrededor terminan siendo dos caras de la misma moneda.

Finalmente, usar Wireshark me demostró que las herramientas que pueden parecer intimidantes al principio se vuelven mucho más manejables luego de un primer uso simple y guiado. Además, también pude entender un poco de por qué esta herramienta es tan importante para el curso. En conjunto, creo que el laboratorio me deja una buena base para el curso, con ganas de profundizar tanto en los protocolos como en las herramientas que se usan para analizarlos.

Referencias

- [1] Mozilla Developer Network, *HTTP messages*. [En línea]. Disponible en: <https://developer.mozilla.org/en-US/docs/Web/HTTP/Guides/Messages>. [Accedido: 13-jul-2026].
- [2] GeeksforGeeks, *ifconfig command in Linux with Examples*. [En línea]. Disponible en: <https://www.geeksforgeeks.org/linux-unix/ifconfig-command-in-linux-with-examples/>. [Accedido: 13-jul-2026].
- [3] Stack Overflow, *How to check length of protocols*. [En línea]. Disponible en: <https://stackoverflow.com/questions/24197371/how-to-check-length-of-protocols>. [Accedido: 13-jul-2026].
- [4] Unwired Learning, *Wireshark and tcpdump: A Practical Guide to Network Traffic Analysis*. [En línea]. Disponible en: <https://unwiredlearning.com/blog/wireshark-tcpdump-guide>. [Accedido: 13-jul-2026].